High Performance Programming [COMTEK3, & ESD3]
Lecture 1: Parallel Processing - Communication and Data Structures 02/2025

Instruction: *The exercise problems are designed to aid your understanding of the concepts covered during the lecture. They are intended to be discussed and solved in a group of* $3-6$ *students. The most important part of the exercise is the* ***learning process****. It is strongly recommended that you avoid the usage of LLMs such as chatGPT and similar AI models to generate solutions* ***without trying to solve the problems first****.*

Exercise 1: **Network Models**

Given a network model with 8 processors connected in a hypercube topology:

(a) Represent the topology as a graph, $G = (V, E)$, labelling each processor.

(b) Calculate the maximum number of steps required for communication between any two processors.

(c) Compare this with the communication steps required in a ring topology with the same number of processors.

Solution: (a). A hypercube topology for $8 = 2^3$ processors is a 3-dimensional cube. The vertices are labeled as binary numbers from 000 to 111, and two vertices are connected if their binary labels differ by exactly one bit. See sketch of 3-cube on the lecture slides.

(b) The maximum distance between two nodes in a hypercube is equal to the dimension of the cube. Thus, for 8 processors (3-cube), the maximum number of steps is:

$$\text{Maximum Steps} = \log_2(8) = 3.$$

(c). In a ring topology, each processor is connected to two neighbors. To communicate between any two processors in the worst case, $\left\lfloor \frac{p}{2} \right\rfloor$ steps are needed, where $p = 8$:

$$\text{Maximum Steps} = \left\lfloor \frac{8}{2} \right\rfloor = 4.$$

Comparison: The hypercube is more efficient for communication as it requires fewer steps (3) compared to the ring (4).

Exercise 2: **Communication Cost**

Assume a message of size $M = 100$ needs to be transferred across a mesh topology with 16 processors (i.e., 4×4 grid). The following parameters are given:

- $t_s = 2$ units (message preparation time)
- $t_w = 1$ unit (unit transfer time)

(a) Derive the communication cost for one-to-all broadcast on this topology.

(b) Analyze how the cost changes when the topology is switched to a hypercube.

Solution:

(a) For one-to-all broadcast on a mesh topology, the communication cost is:

$$T_{comm} = 2t_s(p-1) + t_w M(p-1).$$

Substituting $t_s = 2$, $t_w = 1$, $M = 100$, and $p = 16$:

$$T_{comm} = 2(2)(16-1) + (1)(100)(16-1).$$

$$T_{comm} = 60 + 1500 = 1560 \text{ units}.$$

(b) For a hypercube topology with $p = 16$, the communication cost is:

$$T_{comm} = t_s + t_w M \log_2(p).$$

Substituting $t_s = 2$, $t_w = 1$, $M = 100$, and $\log_2(16) = 4$:

$$T_{comm} = 2 + (1)(100)(4).$$

$$T_{comm} = 402 \text{ units}.$$

Exercise 3: **Broadcast and Reduction**

A distributed system with 16 processors is arranged in a 4×4 mesh topology. Each processor initially holds one element of a vector $V = [3, 6, 1, 5, 7, 4, 2, 9, 8, 3, 5, 1, 4, 2, 6, 7]$.

(a) Design an algorithm to compute the sum of all elements using broadcast and reduction operations.

(b) Explain the steps for the one-to-all broadcast phase to distribute data among all processors.

(c) Calculate the communication cost for the all-to-one reduction phase to collect the result at the first processor.

Solution:

(a) Algorithm for sum computation: To compute the sum of all elements in the vector, V, we emply a one-to-all reduction where each processor contributes its element to the global sum.

1. Initialization: Each processor $p(i, j)$ in the mesh holds one element of V.
2. All-to-One Reduction: Reduce all elements to a single processor (e.g., $p(0, 0)$) by
    - Performing reductions concurrently along the 4 rows first.

- Performing column-wise reduction along the column holding the partial row-wise sums

3. One-to-all Broadcast: not needed for this example but there may be applications where the result needs to be sent to all processors.

(b) See the lecture slides for the steps in one-to-all broadcast.

(c) See the analysis of communication cost of all-to-one reduction on the lecture slides.

Exercise 4: **Scatter and Gather Operations**

A single processor has an array $A = [3, 6, 1, 5, 7, 4, 2, 9, 8, 3, 5, 1, 4, 2, 6, 7]$ with 16 elements. The array needs to be distributed among 4 processors ($p = 4$) in chunks of 4 elements per processor using a scatter operation.

(a) Describe the steps for the scatter operation in a ring topology.

(b) Assume each processor computes the local sum of its elements after the scatter. Write the steps to collect these local sums back at the root processor using a gather operation.

(c) Calculate the communication cost for both scatter and gather operations, assuming: $t_s = 2$, and $t_w = 1$.

Solution:

(a). Scatter operation on a ring topology.

- Step 1: p_0 keeps $[3, 6, 1, 5, 7, 4, 2, 9]$ and sends $[8, 3, 5, 1, 4, 2, 6, 7]$ to p_1.
- Step 2: Concurrent messaging by p_0 and p_1:
  - p_0 keeps $[3, 6, 1, 5]$ and sends $[7, 4, 2, 9]$ to p_2
  - p_1 keeps $[8, 3, 5, 1]$ and sends $[4, 2, 6, 7]$ to p_3

(b). After receiving their respective elements, each processor computes a local sum:

- $p_0 : 3 + 6 + 1 + 5 = 15$
- $p_1 : 8 + 3 + 5 + 1 = 17$
- $p_2 : 7 + 4 + 2 + 9 = 22$
- $p_3 : 4 + 2 + 6 + 7 = 19$

Gather with summation as associative operator.

- Step 1: p_3 sends 19 to p_1 and p_2 sends 22 to p_0.

- Step 2: p_0 and p_1 computes partial sums
- step 3: p_1 sends 36 to p_0
- Step 4: p_0 computes the final sum.

(c) Scatter communication cost: The transfer in step 1 involves 8 elemenst and each transfer involves sending 4 elements, with cost per transfer

$$t_{scatter} = t_s + 8t_w + t_s + 4t_w = 2t_s + 12t_w$$

$$t_{scatter} = 2(2) + 12(1) = 16$$

Gatter communication cost is obtained as

$$T_{gather} = 2 \times (t_s + 1t_w) = 2 \times (2 + 1(1)) = 6$$

Exercise 5: **Data Structures in Parallel Systems**

Match the following data structures with their advantages and challenges in high-performance computing:

- Arrays
- Linked Lists
- Hash Tables
- Graphs

1. Efficient for dynamic memory usage but poor cache performance.
2. Fast lookups but irregular data access patterns in parallel algorithms.
3. Cache-friendly and ideal for vectorization but static in size.
4. High memory costs for adjacency matrix representation but useful for modeling complex relationships.

Solution:

1. **Arrays:** Cache-friendly and ideal for vectorization but static in size.
2. **Linked Lists:** Efficient for dynamic memory usage but poor cache performance.
3. **Hash Tables:** Fast lookups but irregular data access patterns in parallel algorithms.
4. **Graphs:** High memory costs for adjacency matrix representation but useful for modeling complex relationships.